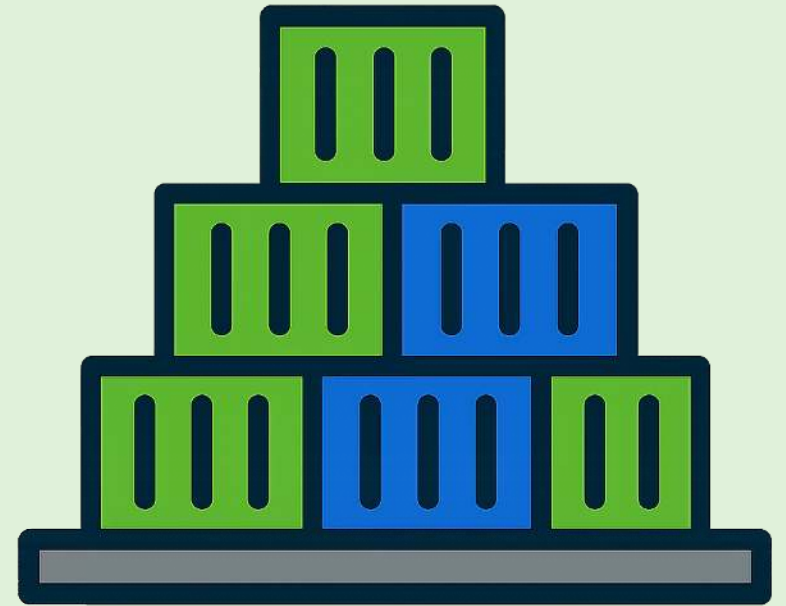


Microservices

REST APIs, Docker Basics & Dockerfiles

Unterrichtseinheit 2/4



Agenda

01

Rückblick UE1

Monolith vs. Microservices

02

REST Grundlagen & API Design

HTTP-Methoden, Statuscodes, URLs

03

Docker Basics

Images, Container, Registry

04

Dockerfiles

Eigene Images bauen

05

Praktische Übungen

5 Hands-on-Aufgaben

Lernziele

Nach dieser Einheit kannst du ...



REST APIs erklären

Prinzipien und Ressourcen benennen



HTTP-Methoden anwenden

GET, POST, PUT, PATCH, DELETE



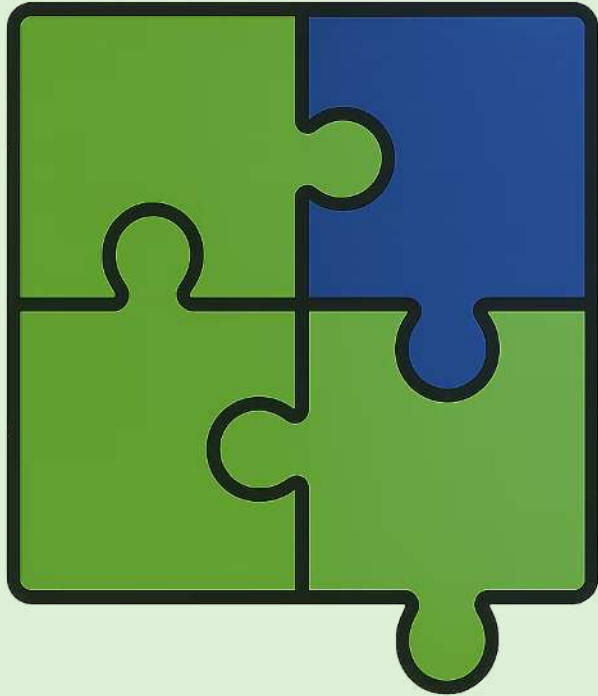
Docker verstehen

Images und Container unterscheiden



Dockerfiles lesen & bauen

Eigene Images erstellen und starten



Rückblick UE1

Was wir wissen

- **Monolith:** Alles in einer Anwendung – schwer skalierbar
- **Microservices:** Kleine, unabhängige Services – jeder für eine Aufgabe
- Services laufen eigenständig und werden separat deployed

Frage zum Einstieg

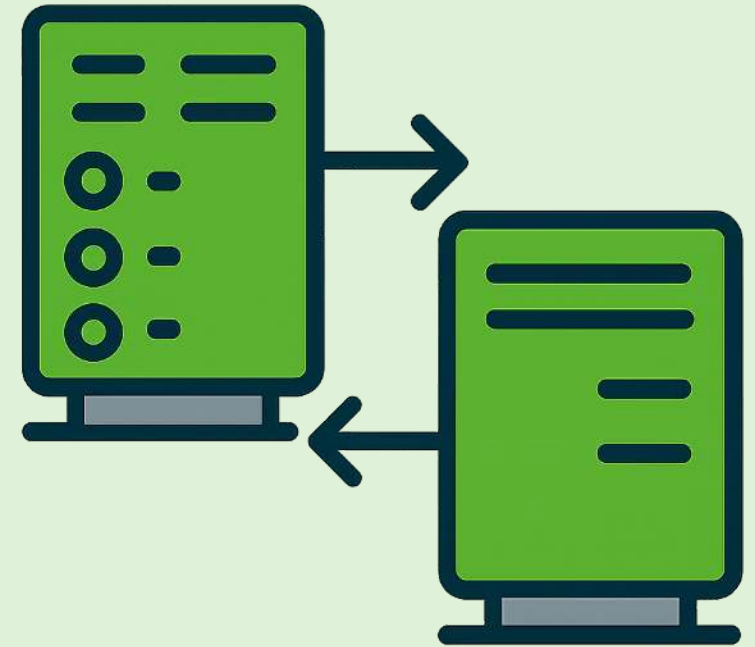
Services arbeiten unabhängig voneinander – aber sie müssen trotzdem miteinander sprechen.

Wie kommunizieren Services miteinander?

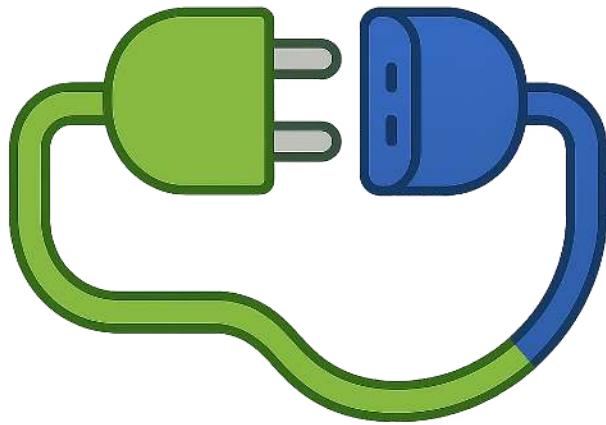
Einstieg: Datenaustausch zwischen Services

Microservices sind nur dann nützlich, wenn sie miteinander kommunizieren können. Ein Bestellservice braucht Daten vom Nutzerservice – ein Frontend braucht Daten vom Backend.

- ❗ Die typische Lösung: **REST APIs über HTTP** – einfach, standardisiert und in jeder Sprache nutzbar.



Was ist eine API?



API = Application Programming Interface

Eine **definierte Schnittstelle** zwischen zwei Systemen. Sie legt fest, welche Anfragen erlaubt sind und welche Antworten zurückkommen.

- Frontend ruft Daten vom Backend ab
- Service A spricht mit Service B
- Mobile App nutzt dieselbe API wie Web

📌 Denk an eine Steckdose: das Interface ist genormt – egal was dahinter steckt.

Was ist REST (Representational State Transfer)?

Ressourcenorientiert

Jede Ressource hat eine eindeutige URL – z. B. `/users/42`

HTTP-Standards nutzen

REST verwendet bewährte HTTP-Methoden und Statuscodes

Stateless

Jede Anfrage enthält alle Informationen – der Server speichert keinen Zustand

Klare URLs

URLs beschreiben Ressourcen, keine Aktionen: `/orders` statt `/getOrders`

Ressourcen in REST

Typische REST-Ressourcen

Ressourcen sind immer **Substantive im Plural** – sie beschreiben Dinge, nicht Aktionen.

/users

/products

/orders

Denkaufgabe: Spotify

Welche Ressourcen würde eine Musik-Streaming-API wie Spotify haben?

→ **Deine Ideen im Chat!**

HTTP-Methoden

REST nutzt die Methoden des HTTP-Protokolls, um festzulegen, **was** mit einer Ressource gemacht wird.



Praktische Aufgabe: HTTP-Methoden zuordnen

Ordne die richtige Methode zu

1. Nutzer löschen
2. Produkte anzeigen
3. Bestellung anlegen
4. Profil ändern

→ Deine Antworten im Chat!

Zur Erinnerung

GET

POST

PUT / PATCH

DELETE

HTTP-Statuscodes

Jede API-Antwort enthält einen **Statuscode**, der das Ergebnis der Anfrage beschreibt.



200 OK

Anfrage erfolgreich – Daten werden zurückgegeben



201 Created

Ressource wurde erfolgreich erstellt



404 Not Found

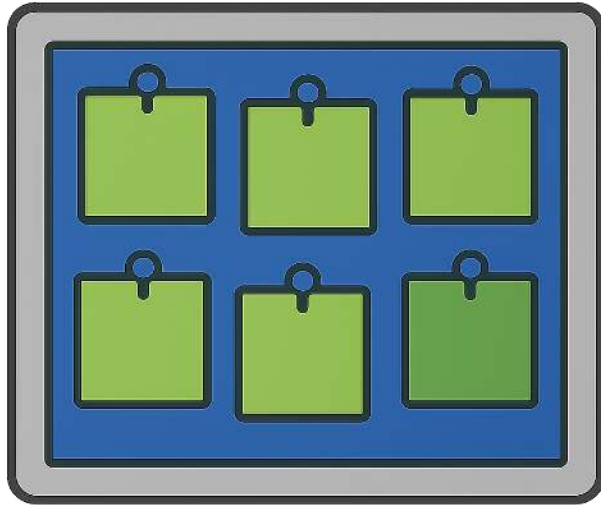
Ressource existiert nicht unter dieser URL



500 Internal Server Error

Serverfehler – etwas ist intern schiefgelaufen

API Design Aufgabe: Notizen-App



Deine Aufgabe

Entwirf die REST API für eine einfache Notizen-App. Definiere für jede Aktion die passende **HTTP-Methode + URL**.

- Alle Notizen anzeigen
- Eine neue Notiz anlegen
- Eine einzelne Notiz löschen

📄 Beispiel: `GET /notes` → alle Notizen abrufen

→ Methode + URL im Chat!

Die API läuft – aber nur lokal

Deine REST API funktioniert auf deinem Rechner. Aber läuft sie auch auf dem Rechner deines Kollegen? Auf dem Server? In der Cloud?

„Works on my machine“ – das häufigste Problem in der Softwareentwicklung.



Was ist Docker?



Software einpacken – überall starten

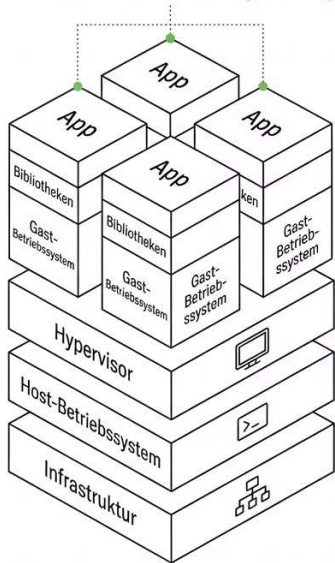
Docker verpackt eine Anwendung zusammen mit allem, was sie braucht, in einen **Container**.

- **App** – dein Code
- **Laufzeit** – z. B. Node.js, Python
- **Abhängigkeiten** – Bibliotheken, Konfiguration

✓ Der Container läuft überall gleich – egal ob lokal, auf dem Server oder in der Cloud.

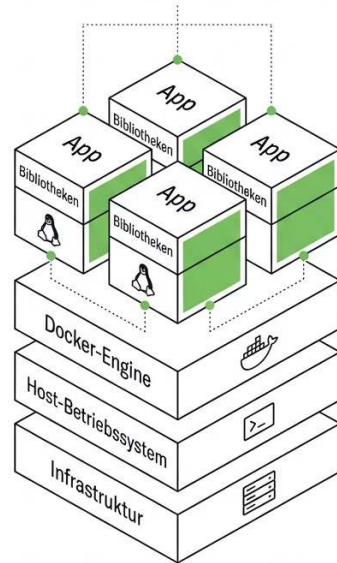
Docker vs. Virtuelle Maschine

Virtuelle Maschine (VM)



Schwer und langsam;
vollständiges Betriebssystem
pro VM.

Docker Container



Leichtgewichtig und schnell;
teilt Host-Kernel, nur App
und Abhängigkeiten.

Vorteile von Docker

Schneller Start

Container starten in Sekunden,
VMs in Minuten

Kleiner Footprint

Kein vollständiges OS – deutlich
weniger Speicherbedarf

Portabel

Läuft auf jedem System mit Docker-Installation

Docker-Begriffe



Image

Die **Vorlage** – eine unveränderliche Bauanleitung für einen Container. Wird einmal gebaut, beliebig oft gestartet.



Container

Ein **laufendes Image** – die echte, ausführbare Instanz. Mehrere Container können dasselbe Image nutzen.



Registry

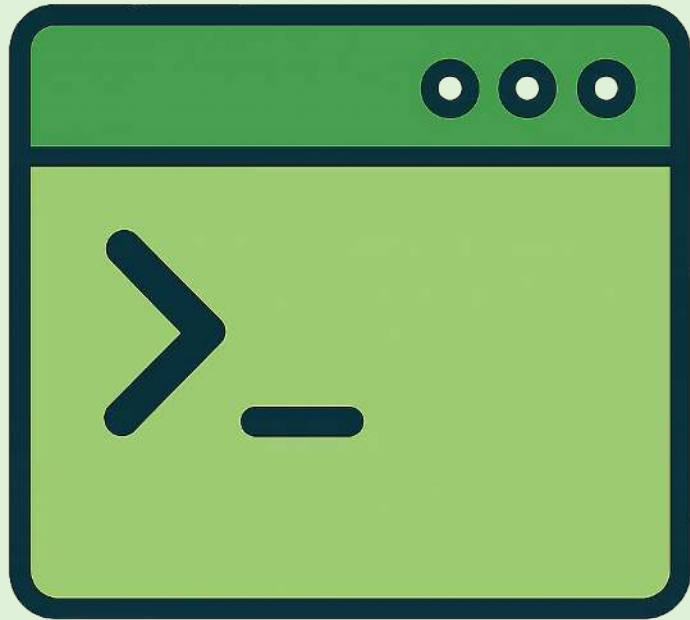
Der **Speicherort** für Images – z. B. Docker Hub. Von dort werden Images heruntergeladen.

Der erste Docker-Start



Bevor du mit Containern arbeiten kannst, muss Docker auf deinem System installiert sein. Docker Desktop ist die einfachste Lösung für Windows, macOS und Linux und enthält alle notwendigen Tools.

1. Lade [Docker Desktop](#) für dein Betriebssystem herunter.
2. Führe die Installation gemäß den Anweisungen aus.
3. Starte Docker Desktop und vergewissere dich, dass die Docker-Engine läuft (grünes Symbol in der Taskleiste/Menüleiste).



Der erste Docker-Start

Befehl

```
docker run hello-world
```

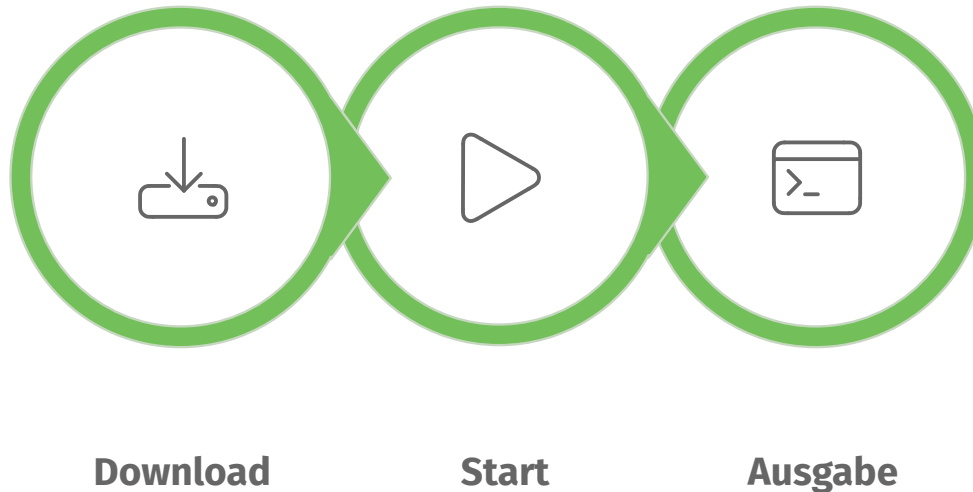
ⓘ Was passiert, wenn du diesen Befehl ausführst? Notiere deine Vermutung!

Repo mit begleitenden Übungen:

<https://github.com/tobias-delp/docker>

Was passiert hinter den Kulissen?

Was passiert?



Hinter den Kulissen

Docker sucht zunächst lokal nach dem Image. Wenn es nicht gefunden wird, lädt Docker es automatisch aus dem Docker Hub herunter.

i Das ``hello-world``-Image ist minimal – perfekt für den ersten Test ohne eigene Konfiguration.

Ersten Container starten

Aufgabe

1. Starte `hello-world` mit Docker
2. Starte einen `nginx`-Container auf Port 8080
3. Öffne `http://localhost:8080` im Browser
4. Erkläre in 2 Sätzen: Was wurde heruntergeladen und was läuft jetzt lokal?

Befehle

```
docker run hello-world
```

```
docker run -p 8080:80 nginx
```

 `-p 8080:80` verbindet Port 8080 auf deinem Rechner mit Port 80 im Container.

Container untersuchen

Aufgabe

1. Liste alle laufenden Container auf
2. Finde Name oder ID des nginx-Containers
3. Stoppe den Container
4. Starte ihn erneut im Hintergrund
5. Zeige die Logs des Containers an

Hilfreiche Befehle

```
docker ps
```

```
docker stop <name>
```

```
docker run -d -p 8080:80 nginx
```

```
docker logs <name>
```

 `-d` = Detached Mode – der Container läuft im Hintergrund.

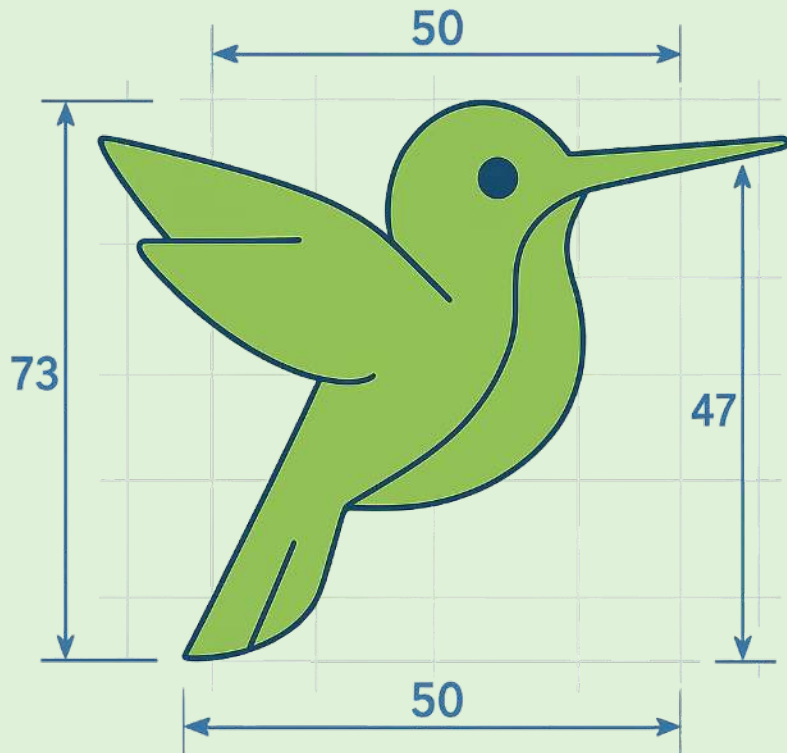
Reflexion

Woran erkennt man, ob ein Container läuft?

`docker ps` zeigt alle aktiven Container – mit ID, Name, Status und Port-Mapping.

Warum ist `-d` im Entwickleralltag praktisch?

Der Container läuft im Hintergrund – das Terminal bleibt frei für weitere Befehle und Logs können jederzeit mit `docker logs` abgerufen werden.



■ 73bf5a ■ 50ftaf31
■ 50ar 31

Eigene Images – wie geht das?

Bisher haben wir fertige Images wie `nginx` oder `hello-world` verwendet. Aber was, wenn wir unsere **eigene Anwendung** in einen Container packen wollen?

❗ Die Antwort: ein **Dockerfile** – die Bauanleitung für dein eigenes Image.

Was ist ein Dockerfile?



Bauanleitung für ein Image

Ein Dockerfile ist eine einfache **Textdatei**, die Schritt für Schritt beschreibt, wie ein Docker-Image aufgebaut wird.

- **Basisimage** definieren (z. B. nginx, node, python)
- **Dateien kopieren** in den Container
- **Startbefehl** festlegen

📄 Wie ein Rezept: Zutaten + Schritte = fertiges Gericht. Hier: Befehle + Dateien = fertiges Image.

Einfaches Dockerfile-Beispiel

Das Dockerfile

```
FROM nginx  
COPY ./usr/share/nginx/html
```

Ergebnis

Deine eigene Website läuft im nginx-Container – ohne separate Webserver-Installation.

Was passiert Zeile für Zeile?

1

FROM nginx

Startet mit dem offiziellen nginx-Image als Basis

2

COPY ./usr/share/nginx/html

Kopiert alle lokalen Dateien in das Webverzeichnis des Containers

Eigene HTML-Seite containerisieren

Aufgabe

1. Erstelle eine `index.html` mit eurem Teamnamen
2. Lege ein Dockerfile für nginx an
3. Kopiere die HTML-Datei in das richtige Verzeichnis
4. Baue das Image mit dem Namen `team-page`
5. Starte einen Container auf Port 8081 und prüfe im Browser

📄 **Erweiterung:** Ändert `index.html`, baut erneut – wann ist die Änderung sichtbar?

Startpunkt & Befehle

```
FROM nginx  
COPY index.html /usr/share/nginx/html/index.html
```

```
docker build -t team-page .
```

```
docker run -p 8081:80 team-page
```

Wichtige Dockerfile-Befehle

1

FROM

Legt das **Basisimage** fest – z. B.
`node:20`, `python:3.11`

2

COPY

Kopiert **Dateien** vom Host in den Container

3

WORKDIR

Setzt das **Arbeitsverzeichnis** im Container

4

RUN

Führt Befehle während des **Build-Vorgangs** aus – z. B. `npm install`

5

CMD

Legt den **Startbefehl** fest, der beim Container-Start ausgeführt wird

Und mehr unter <https://docs.docker.com/reference/dockerfile/>

Beispiel: Node.js Dockerfile

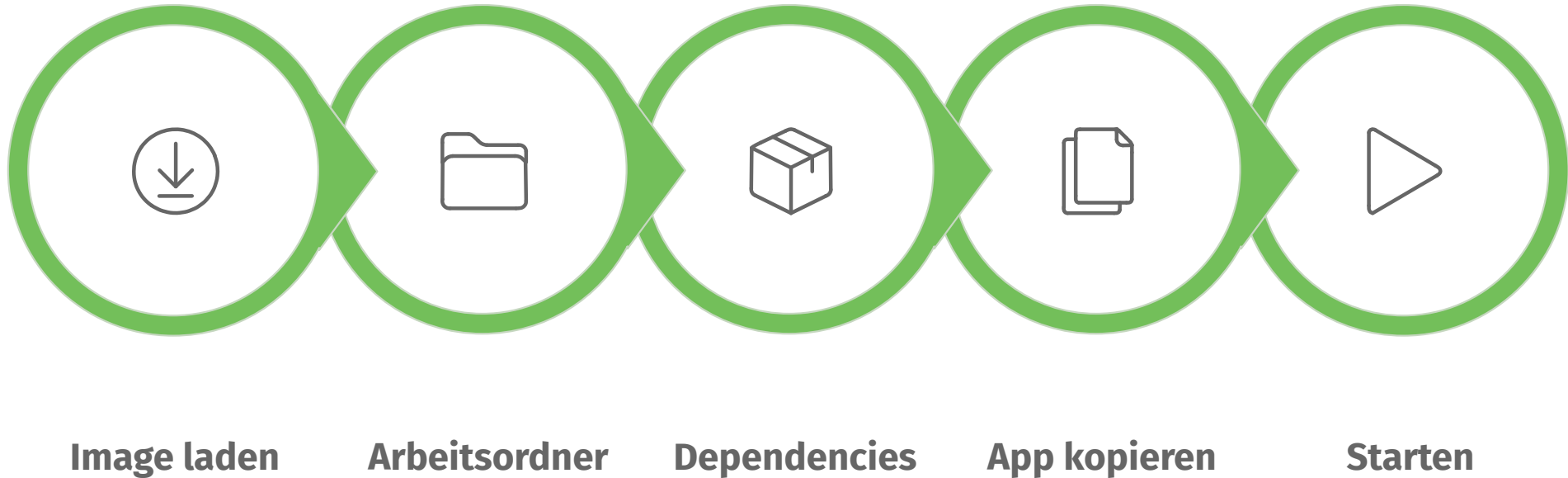
Das vollständige Dockerfile

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["npm", "start"]
```

Warum diese Reihenfolge?

- `package*.json` zuerst kopieren – Docker kann den Installations-Layer cachen
- `npm install` läuft nur neu, wenn sich die Abhängigkeiten ändern
- Restlicher Code wird danach kopiert – schnellere Builds

Erklärung: Schritt für Schritt



Durch das gezielte Vorab-Kopieren von `package*.json` nutzt Docker seinen Layer-Cache optimal – unnötige Neuinstallationen werden vermieden.

Ein Image bauen

Der Build-Befehl

```
docker build -t myapp .
```

Was bedeutet das?

docker build

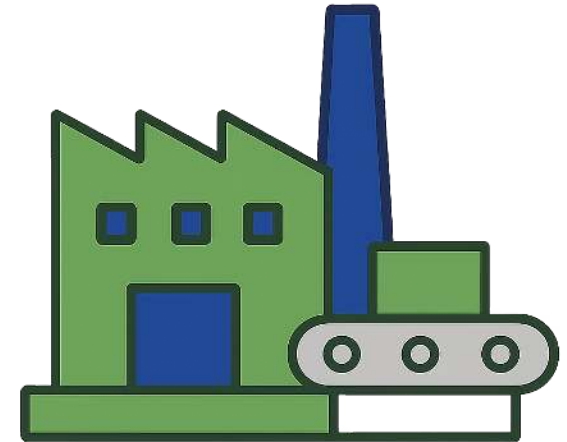
Startet den Build-Prozess

-t myapp

Gibt dem Image den Namen `myapp`

.

Nutzt das aktuelle Verzeichnis als Build-Kontext



Container starten

Befehl

```
docker run -p 3000:3000 myapp
```



`-p 3000:3000` verbindet Port 3000 auf deinem Rechner mit Port 3000 im Container.

Danach erreichst du deine App unter `http://localhost:3000` im Browser.

Image → Container

Das gebaute Image `myapp` wird jetzt als laufender Container gestartet.

Du kannst dasselbe Image mehrfach starten – jeder Container läuft isoliert.

Dockerfile lesen und verbessern

Gegeben (Python/Flask)

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install flask
CMD ["python", "app.py"]
```

Aufgabe (20 min)

1. Beschreibe jede Zeile
2. Nenne 2 mögliche Probleme oder Verbesserungen
3. Welche Datei fehlt noch sinnvollerweise?

Leitfragen

Abhängigkeiten effizient?

requirements.txt
verwenden – erst
Dependencies, dann Code
kopieren

Image-Größe?

.dockerignore verhindert,
dass unnötige Dateien ins
Image gelangen

Port?

Wenn app.py Port 5000 nutzt, muss er beim docker run
freigegeben werden

Häufige Fehler beim Dockerfile

Falscher Port

App läuft auf Port 5000, aber `-p` gibt Port 3000 frei → kein Zugriff



Dateien nicht kopiert

`COPY` vergessen → App findet ihre Dateien im Container nicht



CMD fehlt

Container startet, aber tut nichts – kein Startbefehl definiert



Große Images

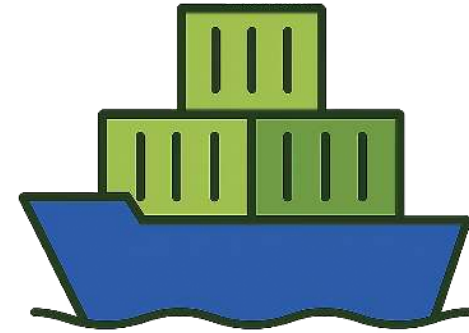
Alle Dateien kopiert ohne `.dockerignore` → unnötig großes Image



Microservices + Dockerfiles

Ein Dockerfile pro Service

In einer Microservices-Architektur bekommt jeder Service sein **eigenes Dockerfile** – und damit sein eigenes Image.



user-service

payment-service

order-service

 Jedes Image enthält nur das, was der jeweilige Service wirklich braucht – kein unnötiger Ballast.

Transfer zu Microservices

Szenario

Ein System besteht aus drei Komponenten:

- **frontend**
- **user-service**
- **order-service**

Aufgaben in Kleingruppen

1. Welche Services brauchen ein eigenes Dockerfile?
2. Welche Dateien und Abhängigkeiten gehören in welches Image?
3. Warum ist ein gemeinsames großes Image für alle Services keine gute Idee?



MICROSERVICES MICROSERVICES MICRDSERVICES

Transfer zu Microservices – Auflösung

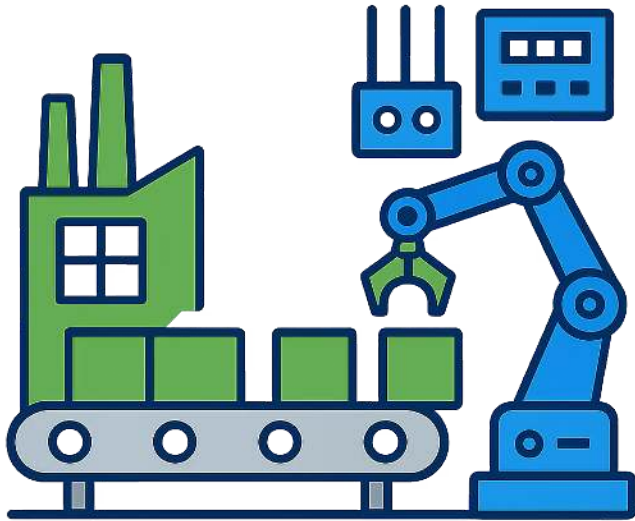
Erwartete Erkenntnisse

→ Jeder Service wird unabhängig gebaut und deployed

→ Images enthalten nur, was der Service braucht

→ Das unterstützt CI/CD und schnelle Deployments

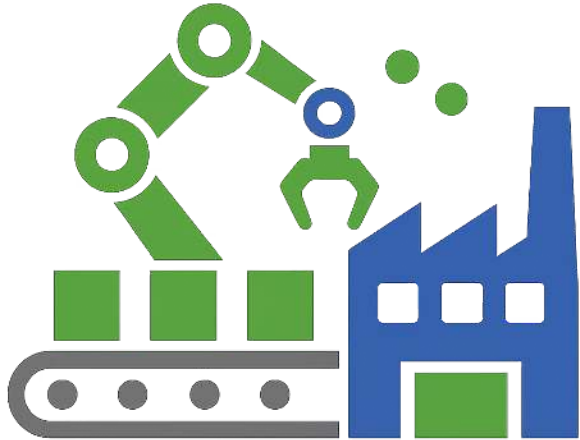
Diskussion: Dockerfiles & CI/CD



Warum sind Dockerfiles wichtig für CI/CD Pipelines?

→ Überlege kurz und teile deine Antwort im Chat!

Dockerfiles & CI/CD – Auflösung



Warum sind Dockerfiles wichtig für CI/CD Pipelines?

In modernen CI/CD-Pipelines (z. B. GitHub Actions, GitLab CI) wird bei jedem Commit automatisch ein neues Image gebaut und deployed.

- Builds sind **reproduzierbar** – immer gleiche Umgebung
- Kein „funktioniert nur bei mir“-Problem
- Rollbacks: einfach auf altes Image zurückwechseln

Zusammenfassung

Das haben wir heute erarbeitet:

REST APIs

HTTP-Methoden, Statuscodes, ressourcenorientierte URLs

Docker Basics

Images, Container, Registry – und der Unterschied zwischen ihnen

Container starten

`docker run`, Port-Mapping, Detached Mode, Logs

Dockerfiles

FROM, COPY, RUN, CMD – eigene Images bauen und starten



Ausblick: Unterrichtseinheit

3

1

Multi-Service Umgebung

Mehrere Container gemeinsam betreiben

2

Docker Compose & Portainer

Services koordinieren und verwalten

3

Mehr Praxis

Eigene Microservices-Umgebung aufbauen

OPTIONAL

Hausaufgabe

Aufgabe

Erstelle lokal eine kleine Webanwendung oder statische HTML-Seite und liefere dazu:

- ☐ Eine `index.html` oder `app.py`
- ☐ Ein passendes `Dockerfile`
- ☐ Die Befehle zum Bauen und Starten

✓ **Bonus:** Dokumentiere in 3–4 Sätzen, welche Probleme Docker für Entwicklerteams löst.

Ziel der Übung

Du sollst den gesamten Workflow selbstständig durchführen:

- Anwendung erstellen
- Dockerfile schreiben
- Image bauen
- Container starten & testen



Viel Erfolg!

Fragen? Jetzt ist der richtige Moment. 🚀